



南開大學
Nankai University

计算机学院

深度学习及应用（高阶课）实验报告

卷积神经网络算法

姓名：申健强

学号：2313119

专业：计算机科学与技术

2026 年 6 月 29 日

目录

1 实验目的与要求	2
2 数据集与训练设置	2
3 模型结构与原理	2
3.1 Baseline CNN	2
3.2 ResNet	3
3.3 DenseNet	3
3.4 MobileNet	3
3.5 Res2Net	4
4 实验结果	4
5 分析与讨论：模型训练过程的差异	6
6 扩展：Res2Net 的优势与劣势	8
7 结论	8

1 实验目的与要求

CNN 是一种经典的深度学习网络，但网络表现不仅取决于深度与参数规模，更取决于它的连接方式：朴素地堆叠卷积层会随深度增加出现梯度消失与退化、难以有效训练。本实验的目标是掌握卷积神经网络（CNN）的基本原理，围绕这一目标，我们使用 PyTorch 在 CIFAR-10 上完成以下工作：

- 复现老师提供的原始版本 CNN，给出网络结构与在测试集上的损失、准确率曲线；
- 自行实现 ResNet、DenseNet、MobileNet 三种网络并完成同样的训练与评测；
- 重点分析无跳跃连接的 CNN、ResNet、DenseNet、MobileNet 在训练过程中的差异；
- 扩展实现 Res2Net，并通过实验说明其相对前述网络的优势与劣势。

2 数据集与训练设置

数据集. CIFAR-10 包含 10 类共 60,000 张 32×32 彩色图像，其中训练集 50,000 张、测试集 10,000 张。由于 CIFAR-10 未单独划分验证集，我们沿用惯例，以 10,000 张测试集作为验证/评测集，后文验证集曲线由此得到。训练集采用标准数据增强：RandomCrop(32, padding=4) 与 RandomHorizontalFlip；训练、测试均按 CIFAR-10 的通道均值、标准差进行归一化。

训练协议. 为保证对比公平，全部采用相同的训练设置：优化器 SGD（动量 0.9，权重衰减 5×10^{-4} ），学习率 0.01，损失函数为交叉熵，批大小 128，训练 30 个 epoch，固定随机种子 42。我们将五个模型统一定义在 models.py，数据/训练/评估工具在 train_utils.py，可视化与对比在 notebook 中进行展示。

实验环境. PyTorch + RTX4060。

3 模型结构与原理

五个网络的结构概览如表 1 所示除基线 CNN 外，其余网络均使用 3×3 卷积作为 CIFAR 适配的 stem（步长 1、不接 maxpool，以免在 32×32 的小图上过早下采样），分类头前统一使用全局平均池化。

表 1: 模型结构概览对比

模型	主体结构	核心思想	参数量
BaselineCNN	2 个卷积 + 3 个全连接	无跳跃连接（对照基线）	0.062 M
ResNet18	4 stage $\times$ 2 BasicBlock	残差连接 $y = F(x) + x$	11.17 M
DenseNet	4 DenseBlock(6) + 3 Transition, $k=12$	密集连接（特征拼接复用）	0.246 M
MobileNet	13 个深度可分离卷积块	深度可分离卷积（轻量化）	3.22 M
Res2Net	4 stage $\times$ 2 Bottle2neck, $scale=4$	块内多尺度分组残差	13.84 M

3.1 Baseline CNN

该网络为最朴素的卷积结构（2 个卷积 + 3 个全连接），无批归一化、无跳跃连接，其 `print(net)` 输出如下¹：

¹注：为保证报告篇幅与美观度在报告中我们尽量只展示代码中最核心的部分或者其思想具体实现可以参考我们提交的代码

```

1 BaselineCNN(
2     (conv1): Conv2d(3, 6, kernel_size=(5, 5), stride=(1, 1))
3     (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
4     (conv2): Conv2d(6, 16, kernel_size=(5, 5), stride=(1, 1))
5     (fc1): Linear(in_features=400, out_features=120, bias=True)
6     (fc2): Linear(in_features=120, out_features=84, bias=True)
7     (fc3): Linear(in_features=84, out_features=10, bias=True)
8 )

```

3.2 ResNet

ResNet 通过**残差连接**让每个模块学习残差映射并与输入做恒等相加：

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x}, \quad (1)$$

恒等捷径为梯度提供了直通路径，缓解了深层网络的梯度消失与退化问题，使网络能够稳定地加深改进。本实验中核心残差模块实现如下：

```

1 class BasicBlock(nn.Module): # two 3x3 conv+BN; shortcut = 1x1 conv when dims change
2     def forward(self, x):
3         out = F.relu(self.bn1(self.conv1(x)))
4         out = self.bn2(self.conv2(out))
5         out += self.shortcut(x) # residual add (identity shortcut)
6         return F.relu(out)

```

3.3 DenseNet

DenseNet 将每一层与其之前**所有层**在通道维拼接，第 l 层的输入为

$$\mathbf{x}_l = H_l([\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_{l-1}]). \quad (2)$$

这种密集连接带来强烈的**特征复用**，使其能以极少的参数达到较高精度；每层都能直接获得来自损失的梯度，梯度流动顺畅。在实验我们使用 DenseNet-BC，稠密层与过渡层实现如下：

```

1 class _DenseBottleneck(nn.Module): # BN-ReLU-Conv1x1(4k) -> BN-ReLU-Conv3x3(k)
2     def forward(self, x):
3         out = self.conv1(F.relu(self.bn1(x)))
4         out = self.conv2(F.relu(self.bn2(out)))
5         return torch.cat([out, x], 1) # dense concat (channel-wise, not add)
6 # _Transition: 1x1 conv compress + 2x2 avg_pool downsample (described in text)

```

3.4 MobileNet

MobileNet 用**深度可分离卷积**替代标准卷积：先逐通道(depthwise) 3×3 卷积，再用 1×1 (pointwise) 卷积融合通道。相比标准卷积，计算量从 $D_K^2 \cdot M \cdot N \cdot D_F^2$ 降为 $D_K^2 \cdot M \cdot D_F^2 + M \cdot N \cdot D_F^2$ ，大幅减少参数与计算量，是轻量化网络的代表。核心模块实现如下：

```

1 class _DepthwiseSeparable(nn.Module): # depthwise 3x3(groups=cin) + pointwise 1x1, each + BN+ReLU
2     def __init__(self, cin, cout, stride=1):
3         super().__init__()
4         self.conv1 = nn.Conv2d(cin, cin, 3, stride, 1, groups=cin, bias=False) # depthwise
5         self.conv2 = nn.Conv2d(cin, cout, 1, bias=False) # pointwise
6         self.bn1, self.bn2 = nn.BatchNorm2d(cin), nn.BatchNorm2d(cout)

```

```

7 def forward(self, x):
8     return F.relu(self.bn2(self.conv2(F.relu(self.bn1(self.conv1(x))))))

```

3.5 Res2Net

Res2Net 在 bottleneck 内部将通道划分为 $scale$ 组，组间做层级残差：

$$y_i = \begin{cases} x_i, & i = 1 \\ K_i(x_i + y_{i-1}), & 1 < i \leq scale \end{cases} \quad (3)$$

由此在单个模块内获得多种感受野尺度的组合，增强了多尺度特征表达能力。本实验使用 $scale = 4$ 、 $baseWidth = 26$ 的 Bottle2neck，骨架与 ResNet 一致，其前向中的多尺度分组残差实现如下：

```

1 def forward(self, x):                                # Bottle2neck, scale=4
2     residual = x
3     out = self.relu(self.bn1(self.conv1(x)))
4     spx = torch.split(out, self.width, 1)             # split channels into scale groups
5     for i in range(self.nums):
6         sp = spx[i] if (i == 0 or self.stype == 'stage') else sp + spx[i]
7         sp = self.relu(self.bns[i](self.convs[i](sp))) # hierarchical residual
8         out = sp if i == 0 else torch.cat((out, sp), 1)
9     if self.scale != 1:                                # concat the last group
10        out = torch.cat((out, spx[self.nums]), 1)
11    out = self.bn3(self.conv3(out))
12    if self.downsample is not None:
13        residual = self.downsample(x)
14    return self.relu(out + residual)

```

4 实验结果

我们将这几个模型在上述实验设置下训练 30 个 epoch，总体结果如表 2 所示。CNN、ResNet、DenseNet、MobileNet、Res2Net 各自的训练/验证曲线如图 4.1 与图 4.2 中进行展示。

表 2: 各模型总体结果对比

模型	参数量 (M)	最佳准确率 (%)	最终准确率 (%)	训练耗时 (min)
BaselineCNN	0.062	68.72	67.17	13.3
ResNet18	11.17	89.82	89.82	21.9
DenseNet	0.246	87.38	86.64	20.4
MobileNet	3.22	84.60	83.83	12.7
Res2Net	13.84	89.92	89.92	45.5

损失曲线. 图 4.1 给出训练集与测试集的损失随 epoch 的变化。带跳跃/密集连接并使用 Batch-Norm 的四个网络损失下降明显快于基线 CNN，其中 ResNet 与 Res2Net 的训练损失最低。

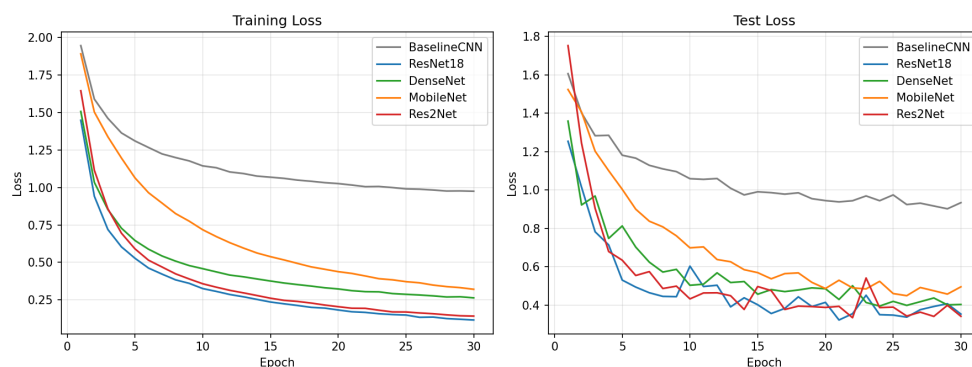


图 4.1: 训练集（左）与测试集（右）损失曲线

准确率曲线. 图 4.2 展示了测试集 Top-1 准确率。ResNet/Res2Net 收敛最快、最终最高；DenseNet 次之但仅用 0.25 M 参数；MobileNet 略低；基线 CNN 始终最低，约 68%。

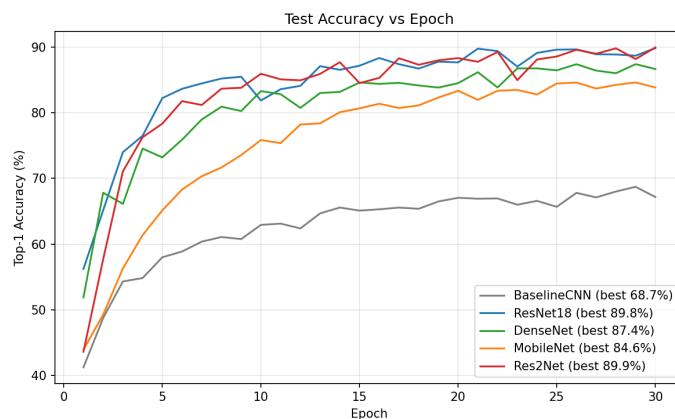


图 4.2: 测试集准确率随 epoch 的变化

各类别准确率. 图 4.3 为各模型在 10 个类别上的准确率热力图。除 Res2Net 外的四个模型均在 cat 类上最难（基线仅 28.5%）；而 Res2Net 把 cat 提升到 84.1%、明显优于其它网络（其自身最难类转为 dog，76.3%），体现了多尺度特征对难样本的帮助。

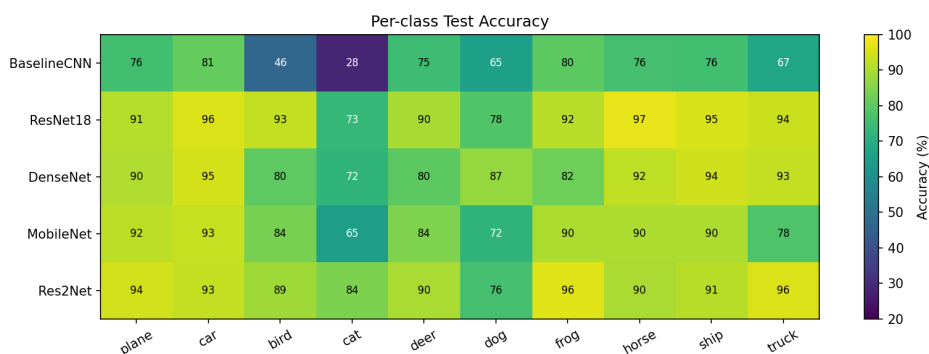


图 4.3: 各模型在每个类别上的测试准确率

模型“看哪里”：Grad-CAM 对比. 为更直观地比较各网络的行为，我们用 Grad-CAM 可视化它们在同一批测试图上的类激活区域（图 4.4）。可以发现，无跳跃连接的基线 CNN 关注区域弥散容

易错误分类；而 ResNet、Res2Net 等带跳跃/密集连接的网络能把注意力紧贴目标主体，预测相对更准、置信度更高。

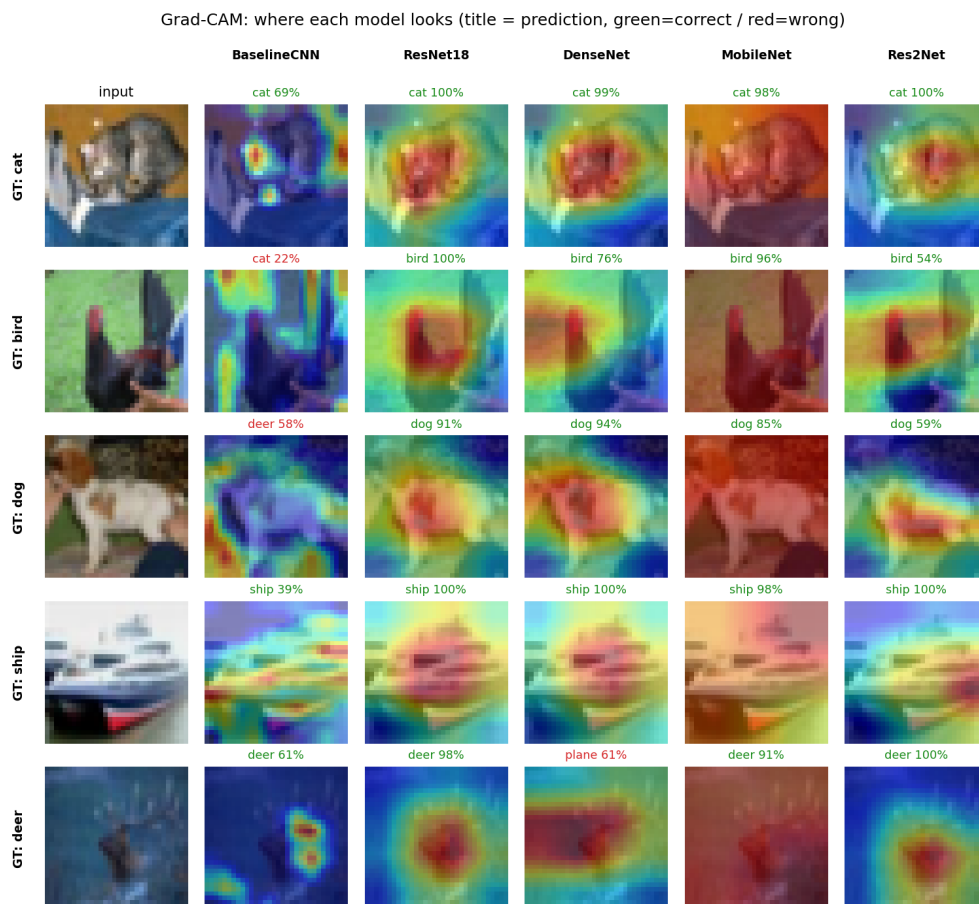


图 4.4: Grad-CAM 对比

5 分析与讨论：模型训练过程的差异

通过训练观察，我们得到如下的一些差异对比

- **无跳跃连接的 CNN (基线)**。结构最浅、无 BatchNorm、参数仅 0.06 M。梯度需逐层穿过普通堆叠卷积，缺乏归一化导致优化困难，损失下降最慢、准确率天花板最低（约 68.7%），且各类别表现极不均衡。它说明：在没有残差/归一化等机制时，单纯堆叠卷积难以高效训练。
- **ResNet**。残差连接 $y = F(x) + x$ 与 BatchNorm 共同作用，提供了梯度直通路径，**收敛最快、训练最稳定**，30 个 epoch 即达到 89.8%，训练损失降到 0.12，曲线平滑无明显波动。
- **DenseNet**。由于密集拼接带来强特征复用，使其以**最少的参数（0.246 M）**获得 87.4% 的高精度，参数效率在若干模型中达到最优；每层直连损失，梯度流动顺畅。但收敛速度略慢于 ResNet，另一方面，通道拼接使显存占用偏高。
- **MobileNet**。深度可分离卷积大幅降低参数与计算量（3.22 M），单位轮次训练快、总耗时最短；但由于缺少残差且表达能力受限，收敛相对较慢、最终精度（84.6%）略低于 ResNet/DenseNet。

梯度流动观察. 我们观察初始化时梯度沿深度的传播可以很好的给上述差异进行解释。由于基线 CNN 仅 5 层、深度不足以暴露深层梯度问题，我们模仿 ResNet 论文式进行同深度对照：在同为 18 层的网络上分别开关 BatchNorm 与跳跃连接，对一个 batch 做一次反向传播，记录各卷积层权重梯度的幅值（图 5.5）。可见：(1) **无 BN、无跳跃**的普通深层网络梯度在靠近输入端急剧衰减，最浅层梯度仅约为最深层的 10^{-4} （梯度消失），底层几乎无法学习；(2) 仅加入 **BatchNorm** 即可把梯度均匀度提升约两个数量级，是缓解初始化期梯度消失的首要因素；(3) 在此之上，**跳跃连接（ResNet）与密集连接（DenseNet）** 进一步使各层梯度更均匀（DenseNet 最高），且随深度增加其作用愈发关键。这解释了带 BN 与跳跃/密集连接的四个网络为何能稳定快速收敛；也说明基线 CNN 的瓶颈主要是**容量小与缺少归一化**

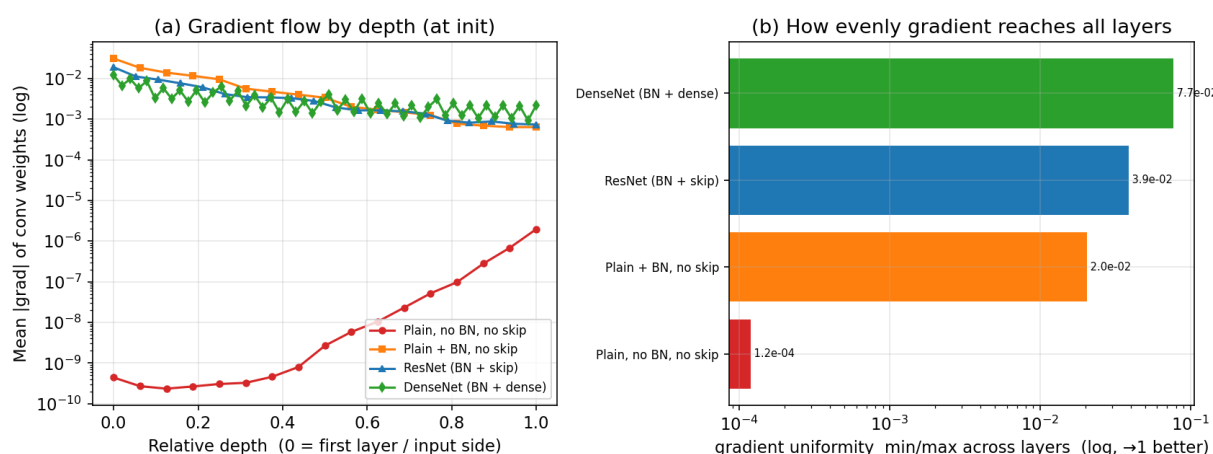


图 5.5: 初始化时逐层梯度幅值（同 18 层深度下隔离 BatchNorm 与跳跃连接）。左：梯度随相对深度的变化（对数纵轴），无 BN 无跳跃者在浅层急剧消失；右：梯度均匀度（各层 min/max，越接近 1 越好）。

收敛速度与计算效率. 从达到目标精度所需 epoch 和计算量—精度两个角度我们可以观察到另一个层面的训练差异（图 5.6）。ResNet/Res2Net 仅 5–6 个 epoch 即达到 80% 测试精度，DenseNet 约 8 个、MobileNet 约 14 个，而基线 CNN 始终无法达到；在计算量 vs 精度的角度，DenseNet 以最少参数与较低 FLOPs 取得 87% 精度、位于效率前沿，MobileNet 计算量最低，而 Res2Net 精度最高但 FLOPs 也最大，约为 ResNet 的 1.4 倍。

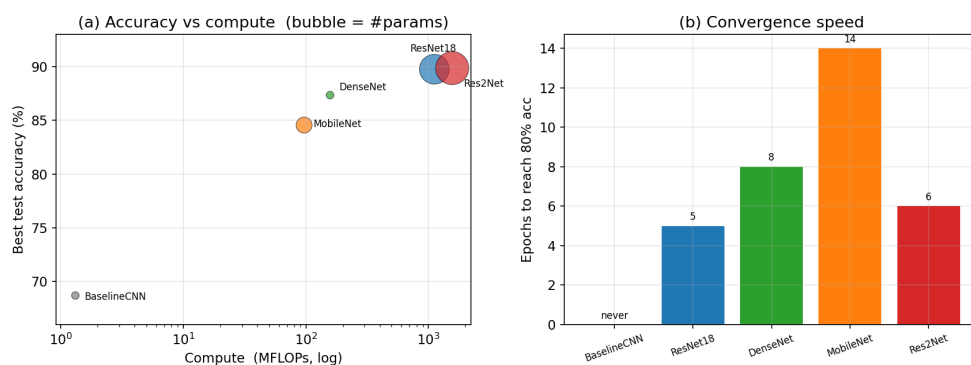


图 5.6: 左：计算量（MFLOPs，对数轴）与最佳准确率，气泡大小表示参数量；右：达到 80% 测试准确率所需的 epoch 数（收敛速度，基线 CNN 始终未达标）。

综合来看，跳跃连接（ResNet）与密集连接（DenseNet）显著改善了梯度传播与收敛行为，是相

对基线 CNN 提升最关键的因素；从轻量化的角度来看 MobileNet 可以在牺牲少量精度的条件下换取轻量化。

6 扩展：Res2Net 的优势与劣势

优势. Res2Net 在模块内构造多尺度感受野，获得了**全场最高的总体准确率 (89.92%)**，并在最难的 cat 类上达到 84.1%、显著优于同级 ResNet 的 73.3%，说明多尺度表达对多尺度/难样本确有帮助。

劣势. 其代价是**参数量最大 (13.84 M)**、**训练最慢 (45.5 min, 约为 ResNet 的两倍)**；而在 CIFAR-10 这种低分辨率任务上，其总体精度相比 ResNet 仅高出约 0.1 个百分点，边际收益有限。

我们把 Res2Net 与 ResNet 的逐类准确率做差 (图 6.7) 可以看到，二者的差异并非全面的，而更像一次性能的**重分配**：Res2Net 在最难的 cat (+10.8 个百分点)、frog、plane 等类别上明显提升，却在 horse (-7.4)、bird 等类别上有所下降，整体仅净增约 0.1 个百分点。多尺度感受野改变了模型所擅长的样本类型，正是其优势与劣势并存的直观体现。

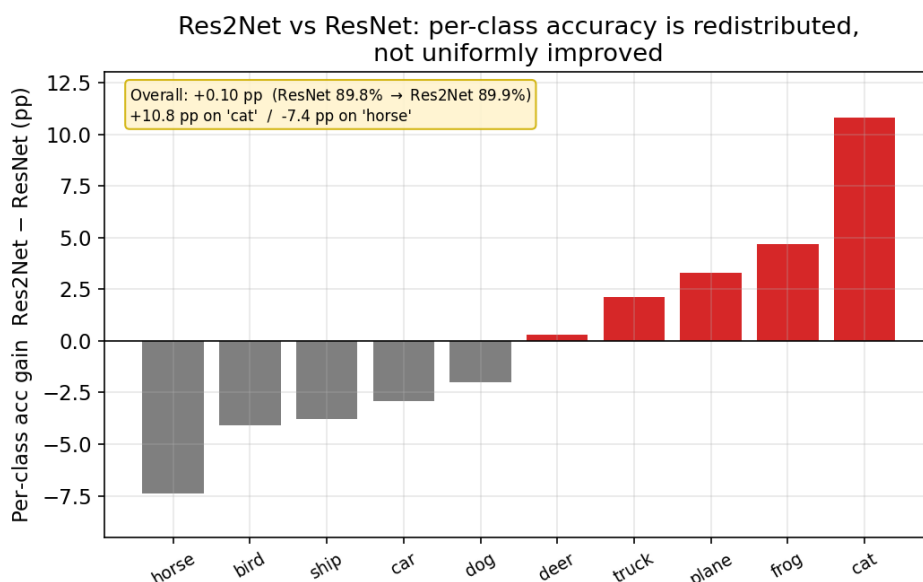


图 6.7: Res2Net 与 ResNet 的逐类准确率之差 (红 = Res2Net 更高)。整体净增仅约 0.1 个百分点，但在难类 cat 上 +10.8、在 horse 上 -7.4

7 结论

综上回顾本次实验，最直观的收获其实不是哪个网络精度最高，而是为什么有的网络好训练、有的不好训练。一开始我们以为基线 CNN 表现差只是因为它浅、参数少；但把它和同为 18 层的网络放在一起、逐层去看初始化时的梯度幅值，才发现真正的瓶颈是缺少归一化——仅 BatchNorm 一项就把梯度均匀度拉高了约两个数量级，而残差与密集连接是在这之上锦上添花，让深层网络的梯度进一步贯通。换句话说，把网络堆深并不会自动带来收益，能不能让梯度顺畅地流回浅层，才是这些结构设计真正在解决的问题

另一点印象比较深的是 Res2Net。它确实拿到了全场最高的总体精度，但比 ResNet 也只高出约 0.1 个百分点，代价却是参数翻倍、训练时间接近两倍。更有意思的是，逐类来看它并非是全面更强，

而是在 `cat` 这类难样本上大幅提升、却在 `horse` 等类别上有所退步，整体看起来更像一次性能的重新分配。

参考文献

- [1] K. He, X. Zhang, S. Ren, J. Sun. Deep Residual Learning for Image Recognition. *CVPR*, 2016.
- [2] G. Huang, Z. Liu, L. van der Maaten, K. Q. Weinberger. Densely Connected Convolutional Networks. *CVPR*, 2017.
- [3] A. G. Howard et al. MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications. *arXiv:1704.04861*, 2017.
- [4] S. Gao, M. Cheng, K. Zhao, X. Zhang, M. Yang, P. Torr. Res2Net: A New Multi-scale Backbone Architecture. *IEEE TPAMI*, 2021.